

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМ. ТАРАСА ШЕВЧЕНКА ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра мережевих та інтернет технологій

РОЗШИРЕННЯ МОЖЛИВОСТЕЙ POSTGRESQL: КОРИСТУВАЦЬКІ ТИПИ, ФУНКЦІЇ ТА ТРИГЕРИ

Практична робота №4

Черкун Єви Сергіївни

Мета: закріпити знання з розширюваності PostgreSQL. Навчитися створювати користувачські типи даних. Реалізувати власну користувачську функцію або агрегат. Створити тригери для логування змін у базі даних. Автоматично оновлювати пов'язані таблиці чи заповнювати значення. Оновити діаграму бази даних відповідно до виконаних завдань. Перевірити коректність роботи реалізованих об'єктів через виконання тестових SQL-запитів.

Хід виконання роботи:

4.1. Визначення атрибута для користувачького типу

У базі даних **medical_lab** атрибут status у таблиці LabOrder містить чітко визначений набір можливих значень (статусів замовлення). Для покращення структури БД, ми замінимо його на **ENUM**-тип.

1. Створення ENUM-типу

```
CREATE TYPE order_status AS ENUM ('pending', 'in_progress', 'completed',  
'canceled');
```

2. Оновлення структури таблиці

```
ALTER TABLE LabOrder ALTER COLUMN status TYPE order_status USING  
status::text::order_status;
```

4.2 Реалізація функції для підрахунку завершених замовлень

1. **Створення функції:** Функція count_completed_orders() обчислює кількість замовлень із статусом 'completed', зроблених за останній місяць.
2. **Запит до функції:** Для перевірки коректності виконання функції виконується запит, що повертає результат.

```
CREATE OR REPLACE FUNCTION count_completed_orders() RETURNS INT  
AS $$
```

```
BEGIN
```

```
    RETURN (SELECT COUNT(*) FROM LabOrder WHERE status = 'completed'  
AND order_date >= NOW() - INTERVAL '1 month');  
END;  
$$ LANGUAGE plpgsql;
```

3. Використання функції:

```
SELECT count_completed_orders();
```

4.3 Створення тригерів для логування змін

Для відслідковування змін у таблиці LabOrder буде створена таблиця для логування змін і тригер, що автоматично фіксуватиме всі зміни статусу замовлень.

1. **Створення таблиці для логування змін:** Таблиця LabOrder_Log міститиме записи про зміну статусів замовлень, де буде зберігатися ідентифікатор замовлення, старий і новий статус, а також час зміни.

```
CREATE TABLE LabOrder_Log (  
    log_id SERIAL PRIMARY KEY,  
    order_id INT REFERENCES LabOrder(id),  
    old_status order_status,  
    new_status order_status,  
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2. **Створення тригерної функції для логування змін:** Тригерна функція log_laborder_changes() буде викликана після оновлення статусу замовлення і додаватиме інформацію про зміну в таблицю логів.

```
CREATE OR REPLACE FUNCTION log_laborder_changes() RETURNS  
TRIGGER AS $$  
BEGIN  
    INSERT INTO LabOrder_Log (order_id, old_status, new_status)  
    VALUES (OLD.id, OLD.status, NEW.status);  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

3. **Прив'язка тригера до таблиці LabOrder:** Тригер буде виконуватися після кожного оновлення статусу замовлення в таблиці LabOrder, фіксуючи зміни в таблиці логів.

```
CREATE TRIGGER track_laborder_changes  
AFTER UPDATE ON LabOrder  
FOR EACH ROW
```

```
EXECUTE FUNCTION log_laborder_changes();
```

4. **Перевірка роботи тригера:** Після зміни статусу замовлення на 'completed', ми перевіряємо таблицю логів, щоб побачити запис про зміну.

```
UPDATE LabOrder SET status = 'completed' WHERE id = 1;  
SELECT * FROM LabOrder_Log;
```

Очікуваний результат:

log_id	order_id	old_status	new_status	changed_at
1	1	pending	completed	2025-04-02 14:30:00

(1 row)

Висновок:

У результаті виконання лабораторної роботи були реалізовані функція для підрахунку завершених замовлень за останній місяць, таблиця логування змін статусів замовлень та тригер для автоматичного фіксування цих змін. Всі функції і тригери були протестовані, що підтвердило їх правильну роботу. Ці зміни автоматизують процеси управління даними, підвищуючи ефективність і точність роботи з базою даних та зменшуючи ймовірність помилок.